

Data Acquisition and Integration Pipeline

Python-based Multi-Source Data Integration with Large-Data Processing and Schema Drift Detection

1. Problem Formulation

Modern data science applications often receive data from different sources and formats. This project designs a unified data-acquisition pipeline that combines CSV sales data, Excel employee correction data, JSON website logs, and weather data representing an API response. The sales source can contain 5 million records, so the pipeline must avoid loading the complete dataset into memory at once. In addition, the weather API may change its response structure without warning. The system therefore needs mechanisms for validation, integration, large-scale processing, and schema-drift detection.

Objectives

- Acquire data from CSV, Excel, JSON and weather/API sources.
- Clean and standardize the different data formats.
- Integrate the sources into one analytical dataset.
- Process a 5-million-record sales file efficiently using chunks.
- Detect unexpected changes in the weather response schema.
- Handle known field-name changes safely and test the pipeline.

2. Proposed Approach / Methodology

The pipeline follows a simple ETL-style workflow: ingestion, validation, transformation, integration, and output. Sales data acts as the main dataset. Employee information is joined using `employee_id`. Website logs are converted from JSON into a tabular form, their timestamps are converted to dates, and web activity is summarized by employee and date. Weather information is then joined using date and location.

Source	Format	Processing
Sales	CSV	Read normally for sample data; large version processed in 100,000-row chunks
Employees	Excel	Read with Pandas/OpenPyXL and joined using <code>employee_id</code>
Website logs	JSON	Normalized into a DataFrame and aggregated by employee/date
Weather	JSON/API response	Validated against expected schema and joined by date/location

Large Dataset Strategy

The 5-million-row sales CSV is generated deterministically and processed using Pandas chunking. A chunk size of 100,000 records is used. Each chunk is cleaned, duplicate and missing rows are handled, and aggregate values are calculated before the next chunk is loaded. This reduces peak memory usage compared with loading all 5 million records simultaneously. The large CSV is kept outside the GitHub repository because the project instructions require a lightweight repository.

3. Schema Drift Detection and Handling

The expected weather schema contains date, location, temperature, and rainfall. Before integration, the actual response columns are compared with the expected set. Missing and unexpected fields are reported. For known changes, the pipeline applies a controlled alias mapping. For example, an API response containing **temp** and **rain** can be mapped to the standard fields **temperature** and **rainfall**. This prevents a known API change from silently breaking downstream analysis.

Situation	Pipeline response
Expected fields received	Schema validated and processing continues
Known field rename: temp → temperature	Alias mapping standardizes the field
Known field rename: rain → rainfall	Alias mapping standardizes the field
Unknown/missing fields	Schema change is reported for investigation

4. Results and Testing

The implemented Python pipeline successfully loaded the sample CSV, Excel, website JSON, and weather data, integrated them into a unified dataset, and produced the final CSV output. A separate 5-million-record sales dataset was generated and processed in 100,000-record chunks. The schema-drift test was also demonstrated by changing the weather fields from temperature/rainfall to temp/rain. The validation logic detected the changed schema, and the alias-handling logic converted the known names back to the standard schema.

Test / Demonstration	Result
Sample sales ingestion	Passed
Employee Excel ingestion	Passed
Website JSON ingestion	Passed
Weather schema validation	Passed
Changed weather schema detection	Passed
Known changed-field handling	Passed
Automated Pytest checks	3 tests passed

Complexity and Practical Limits

For the large sales source, chunk processing requires approximately $O(n)$ time for a single pass over n records. Peak memory is approximately $O(c)$, where c is the selected chunk size, plus the smaller reference tables used for joins. The approach is practical for large CSV files because memory usage is controlled by the chunk size. For production deployment, additional optimizations such as database storage, indexing, incremental API retrieval, logging, monitoring, retry policies, and data-quality dashboards could be added.

5. Reflection and Future Improvements

The main challenge was integrating different data formats while keeping the process simple and reproducible. Another challenge was handling API schema drift without silently producing incorrect analytical results. The current prototype uses a saved weather response for reproducibility and demonstrates the same validation logic that can be applied to a live API. Future work could add live API

retrieval, response version tracking, automatic quarantine of unknown schemas, alerts, incremental daily updates, and a database-backed analytical layer.

Honesty / Source Note

AI assistance and online documentation were used during development for learning, coding support, and project organization. The submitted implementation was tested and reviewed by the student.